arXiv:2603.03823v3 [cs.SE] 18 Mar 2026

SWE-CI: Evaluating Agent Capabilities in Maintaining Codebases via Continuous Integration

Jialong Chen[1,3,*] ✉, **Xander Xu**[2,*] ✉, **Hu Wei**[2] ✉, **Chuan Chen**[1,†] ✉, **Bing Zhao**[2,†] ✉

[1]Sun Yat-sen University, [2]Alibaba Group, [3]Skylenage

🤗 Hugging Face: https://huggingface.co/datasets/skylenage/SWE-CI

Github: https://github.com/SKYLENAGE-AI/SWE-CI

Abstract

Large language model (LLM)-powered agents have demonstrated strong capabilities in automating software engineering tasks such as static bug fixing. However, in the real world, the development of mature software is typically predicated on complex requirement changes and long-term feature iterations — a process that static, one-shot repair paradigms fail to capture. To bridge this gap, we propose **SWE-CI**, the first repository-level benchmark built upon the Continuous Integration loop, aiming to shift the evaluation paradigm for code generation from static, short-term *functional correctness* toward dynamic, long-term *maintainability*. The key insight is simple: **Maintainability can be revealed by tracking how functional correctness changes over time.** The benchmark comprises 100 tasks, each deriving from a real-world code repository with a development history spanning an average of 233 days and 71 consecutive commits. SWE-CI requires agents to systematically resolve these tasks through dozens of rounds of analysis and coding iterations. SWE-CI provides valuable insights into how well agents can sustain code quality throughout long-term evolution.

1 Introduction

Automating software engineering has long been a central objective in the field of artificial intelligence. In recent years, breakthroughs in large language models (LLMs) have lent substantial momentum into this pursuit — from code completion and test generation to end-to-end program repair, LLM-driven agents have demonstrated capabilities rivaling those of human developers across multiple benchmarks. The concurrent evolution of coding benchmarks has played a pivotal role in this progress, providing both rigorous capability measurement and clear research direction.

At the code generation level, HumanEval [1], MBPP [2] and LiveCodeBench [3] established the single-file synthesis paradigm. At the repository level, SWE-bench [4] introduced the "Issue-to-PR" paradigm, requiring models to generate patches within complete repository contexts. At the agent interaction level, Terminal-bench [5] and τ-bench [6] further broadened the evaluation scope to encompass terminal operations and multi-turn tool-use. Together, these efforts have established a multi-granularity and multi-scenario evaluation ecosystem for code intelligence.

Despite the breadth and depth of this ecosystem, its underlying paradigm exhibits one fundamental limitation: Existing benchmarks almost exclusively focus on measuring the agent's ability to write functionally correct code. However, in the real world, successful software is rarely achieved overnight;

[*]Equal contribution.
[†]Corresponding author.

Preprint. Under review.

it is the result of long-term maintenance. Lehman's Laws reveal that software quality inherently degrades as maintenance progresses [7], while classic literature has established that maintenance activities account for 60% to 80% of total software lifecycle costs [8]. Therefore, there is an urgent need to design new benchmarks to effectively reflect the ability of models to maintain code.

The reason this capacity has long been absent from evaluation is rooted in the prevailing benchmark paradigm itself. From HumanEval and LiveCodeBench to SWE-bench and Terminal-Bench, existing benchmarks universally adopt a snapshot-style protocol: the agent receives a single, complete requirement and produces a one-shot solution. Under this paradigm, an agent that hard-codes a brittle fix and one that writes clean, extensible code may both pass the same test suite — their difference in maintainability is simply invisible. It becomes visible only when the codebase must evolve: new requirements arrive, interfaces change, and modules must be extended. At that point, the cost of earlier design decisions compounds, and an agent that routinely produces poorly structured code will find each successive modification harder, eventually failing to keep pace. This yields a critical insight: **an agent's ability to maintain code can only be revealed through long-term evolution, where the consequences of past decisions accumulate over successive changes.**

Building upon this insight, we propose **SWE-CI** (**S**oft**W**are **E**ngineering – **C**ontinuous **I**ntegration), a novel benchmark designed to evaluate how well agents maintain code across long-term code evolution. SWE-CI comprises 100 tasks, each defined by a *base commit* and a *target commit* from a real-world repository, with an average of 233 days and 71 consecutive commits of authentic evolutionary history in between. SWE-CI employs an Architect–Programmer dual-agent evaluation protocol: starting from the base commit, the agents execute a CI-loop that iteratively generates requirements, modifies source code, and runs tests, with the ultimate objective of passing all tests associated with the target commit. SWE-CI introduces **EvoScore** (**Evo**lution **Score**) as a proxy metric: it measures functional correctness on future modifications, so that agents whose earlier decisions facilitate subsequent evolution score higher, while those that accumulate technical debt see progressively declining performance.

We conduct extensive experiments with a total consumption of more than 10 billion tokens. Results reveal that, despite substantial progress on functional correctness, state-of-the-art models still struggle when tasked with sustaining code quality over extended evolution. We further provide comprehensive, fine-grained analyses of the evaluation results, offering valuable insights into the coding capabilities of LLM-based agents and demonstrating the distinctive diagnostic value of SWE-CI.

2 Measuring the Agent's Ability to Maintain Codebase

2.1 Task formalization

We first establish a unified formalization for agentic coding tasks. Let t denote a single unit test and $\mathcal{T} = \{t_1, t_2, \ldots, t_{|\mathcal{T}|}\}$ the set of all tests we are interested in. Let $\mathcal{C}$ denote the space of codebases and $\mathcal{R}$ the space of requirements. We further define two functions:

- $\mathsf{require}_{\mathcal{T}} : \mathcal{C} \times \mathcal{C} \to \mathcal{R}$, which identifies the functional gap between two codebases with respect to $\mathcal{T}$, and produces a requirement document accordingly;
- $\mathsf{code}_{\mathcal{T}} : \mathcal{R} \times \mathcal{C} \to \mathcal{C}$, which modifies a codebase according to a given requirement and returns an updated codebase.

With these definitions, we observe that many prevailing coding benchmarks [1, 2, 3, 4, 5, 6] follow the *snapshot-based* evaluation paradigm illustrated in Figure 1. In this paradigm, the requirement depends only on the base codebase c_0 and the "golden" codebase c_*, i.e., $r \equiv \mathsf{require}_{\mathcal{T}}(c_0, c_*)$. However, in this paper, we turn to consider an *evolution-based* evaluation paradigm. The requirement in this paradigm is derived from the current codebase dynamically: $r_i = \mathsf{require}_{\mathcal{T}}(c_i, c_*)$, and the codebase is updated accordingly: $c_{i+1} = \mathsf{code}_{\mathcal{T}}(c_i, r_i)$. This iterative loop ensures that the consequences of earlier modifications propagate into subsequent iterations, making the agent's long-term decision quality observable.

2.2 Normalized Change

Most benchmarks treat passing all test cases as the gold standard for functional correctness. However, in software engineering, some features require long-term planning and incremental development

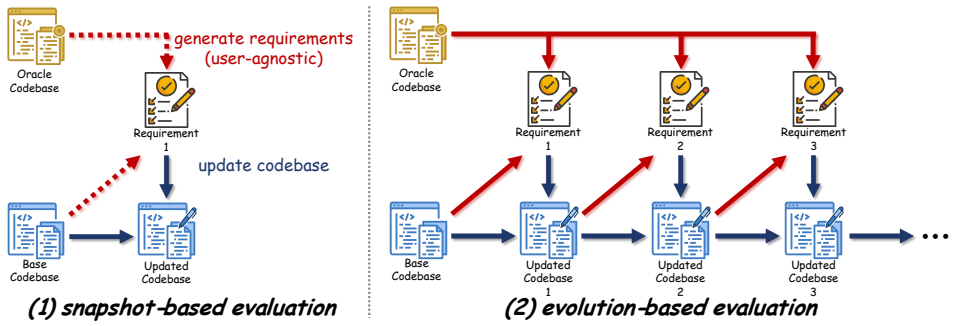

Figure 1: Unlike previous benchmarks, SWE-CI proposes an evolution-based evaluation. The red and blue arrows represent the actions of functions require and code, respectively. Dashed lines indicate processes that are unknown to the user.

rather than a one-shot implementation. Moreover, during evolution it is common for previously passing tests to be inadvertently broken — a phenomenon known as *regression*. We therefore need a finer-grained metric that reflects the current state of a codebase c, rather than a binary pass/fail verdict. To this end, we introduce the **normalized change**.

Let $n(c)$ be the number of test cases that c passes:

$$n(c) = \sum_{t \in \mathcal{T}} \mathbb{I}(t, c), \tag{1}$$

where the indicator function $\mathbb{I}(t, c)$ equals 1 if and only if unit test t passes on codebase c, and 0 otherwise. The normalized change is then defined as:

$$a(c) = \begin{cases} \dfrac{n(c) - n(c_0)}{n(c_*) - n(c_0)} & \text{if } n(c) \geq n(c_0), \\ \dfrac{n(c) - n(c_0)}{n(c_0)} & \text{else.} \end{cases} \tag{2}$$

When the agent improves upon the base codebase ($n(c) \geq n(c_0)$), the numerator is normalized by the total gap $n(c_*) - n(c_0)$, so that $a(c) = 1$ if and only if the agent has closed the gap entirely. When the agent regresses below the baseline ($n(c) < n(c_0)$), the numerator is instead normalized by $n(c_0)$, so that $a(c) = -1$ corresponds to breaking every initially passing test. This asymmetric normalization is deliberate: regardless of how large or small $n(c_0)$ and $n(c_*)$ are, improvements and regressions are always expressed on a unified, comparable scale — i.e., $a(c) \in [-1, 1]$.

2.3 EvoScore

The ISO/IEC 25010 standard defines maintainability as the degree to which software can be modified effectively without introducing defects or degrading existing quality — simply put, more maintainable code is less likely to break future functionality. This yields a simple insight: maintainability can be revealed by tracking how functional correctness changes over time. Guided by this principle, given the sequence of codebases $(c_1, \ldots, c_N)$ obtained from N iterations, we aggregate them into a single scalar, **EvoScore**, via a **future-weighted mean**:

$$e = \frac{\sum_{i=1}^{N} \gamma^i \, a(c_i)}{\sum_{i=1}^{N} \gamma^i} \tag{3}$$

In EvoScore, we set $\gamma \geq 1$ so that later iterations receive *greater* weight. The rationale directly mirrors the ISO definition: a truly maintainable codebase is one that remains easy to modify as evolution progresses. An agent that sacrifices short-term speed for a cleaner, more extensible design will be rewarded over one that rushes to pass early tests but accumulates technical debt that cripples subsequent evolution. When $\gamma = 1$, EvoScore reduces to the *average normalized change*; as γ increases, the metric progressively favors long-term stability over immediate gains.

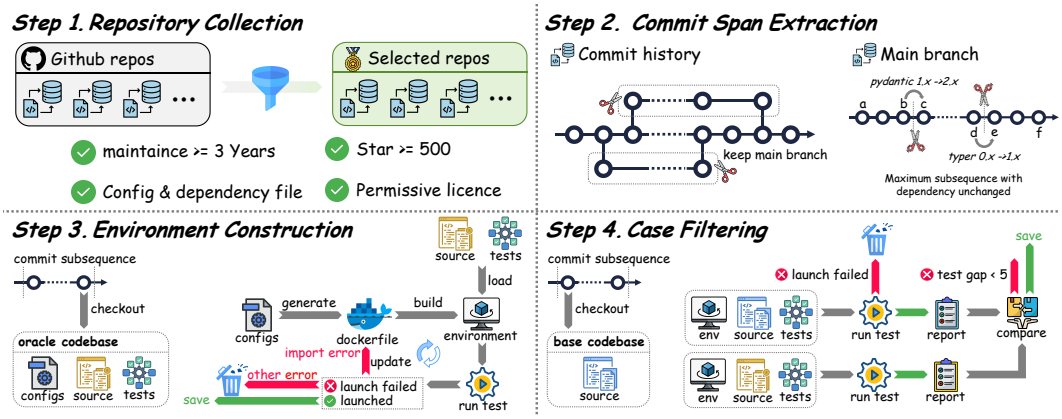

Figure 2: Data curation process of SWE-CI.

3 SWE-CI

3.1 Data curation

As shown in Figure 1, our goal is to obtain a number of base-codebase/oracle-codebase pairs and to let the agent iteratively evolve the former toward the latter, measuring its ability to maintain code throughout this process. Each such pair can be viewed as two chronologically ordered commits within the same repository. Concretely, the construction of SWE-CI is carefully orchestrated as follows:

Step 1: Repository Collection. Unlike SWE-Bench and similar benchmarks that draw exclusively from a handful of well-known open-source prjects, we cast a wider net by searching across all Python repositories on GitHub. We then apply the following filtering criteria: (1) the repository has been actively maintained for at least three years; (2) it has accumulated more than 500 stars; (3) it contains configuration and dependency files (e.g., `pyproject.toml` and lockfiles) as well as a suite of unit tests; and (4) it is released under a permissive license such as MIT or Apache-2.0. After applying these filters, 4,923 repositories remain.

Step 2: Commit Span Extraction. For each surviving repository, we retain only its main branch, reducing the history to a linear sequence of commits. We then compare the dependencies of consecutive commits along this sequence and identify all maximal subsequences within which the dependencies remain unchanged. The two endpoints of every such subsequence naturally form a candidate base/oracle pair. We further discard pairs whose total number of modified lines of code is below 1,000, as such pairs represent insufficient evolutionary distance. This process yields 8,311 candidate pairs.

Step 3: Environment Construction. For each candidate pair, we automatically generate a Dockerfile based on the configuration and dependencies of the oracle codebase and snapshot the resulting runtime environment. We then execute the oracle codebase's unit test suite within this environment to verify its correctness. To improve data retention, we introduce a self-repair mechanism: whenever the test suite fails to launch due to a missing dependency, we detect the failure and dynamically inject the required dependency into the Dockerfile to build a new environment. This mechanism substantially increases the number of viable candidate pairs. Pairs whose failures stem from other reasons are discarded. After this step, 1,458 candidate pairs and their runtime environment snapshots remain.

Step 4: Case Filtering. Finally, we apply three further rounds of filtering to ensure the quality of the final dataset. First, within the runtime environment snapshot constructed in Step 3, we run the oracle codebase's test suite against the base codebase. Any candidate whose tests fail to launch is discarded. Second, we compare the test reports produced by the base and oracle codebases on the same test suite; candidates for which the difference in the number of passing tests is fewer than five are removed. After these two automated filters, 137 candidates remain. In the last round, we rank the

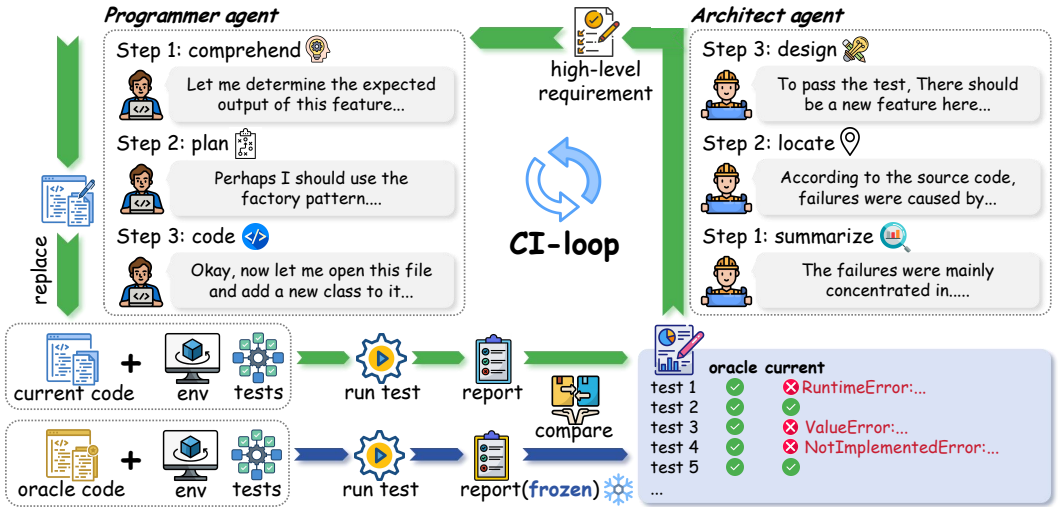

Figure 3: SWE-CI uses an architect-programmer dual-agent workflow to model the continuous integration cycle of professional software teams in the real world.

surviving candidates by their time span and number of intervening commits, and select the top 100 to form the final SWE-CI benchmark.

The final SWE-CI benchmark comprises 100 samples drawn from 68 distinct repositories. On average, each base/oracle pair is separated by 233 days and 71 consecutive commits of real-world development history. In every pair, the transition from the base to the oracle codebase involves at least 500 lines of modified source code, excluding changes to test files. Each sample is shipped with the complete source code and a pre-built Docker environment to ensure reproducibility. These statistics confirm that SWE-CI captures substantial, long-term evolutionary episodes rather than trivial incremental changes.

3.2 Dual-agent evaluation protocol

As described in Figure 1, SWE-CI adopts evolution-based evaluation. To support this setting, we introduce an Architect-Programmer dual-agent protocol. The Architect identifies functional gaps and issues requirements; the Programmer implements them. Their collaboration reproduces the CI loop in real-world development, enabling fine-grained observation of how well agents maintain code.

Architect agent. Based on the test gap between the current code and the oracle code, the Architect is tasked with producing a high-level requirements document in natural language. We prompt the architect to organize its behavior into three steps: ❶ **Summarize**. Architect reviews all failing tests, identifies root causes, and identifies source code files that need further inspection; ❷ **Locate**. Architect examines the source code and attributes failures to concrete deficiencies in the current implementation; ❸ **Design**. Based on these deficiencies, architect devises an improvement plan and produces the final requirements document. Two writing conventions are further imposed on requirement document. ① *Incremental*. The document should contain no more than five of the most urgent requirements, avoiding the pitfall of over-designing in a single iteration. ② *High-level*. the requirements should focus on describing expected behavior of code using natural language, leaving concrete implementation choices to the programmer. The core purpose of these specifications is to ensure that requirement documents meet the needs of real-world continuous integration processes.

Programmer. The programmer's responsibility is to maintain the code according to the requirements document. Programmer behavior is also standardized into three steps: ❶ **Comprehend**. Programmers understand high-level language requirements and translate them into explicit code specifications. ❷ **Plan**. Programmers plan the programming effort required to implement these specifications. ❸ **Code**. Programmers put these plans into practice and try to fulfill the requirements.

In this protocol, the Programmer is driven by the requirements document rather than directly by the test gap — a deliberate design choice that aligns with the rapid iteration philosophy of continuous integration. To this end, the Architect is required to distill the most pressing requirements from the full set of failures, allowing the Programmer to focus on fast, targeted development without being overwhelmed by the full scope of changes.

4 Experiments

4.1 Experiment setting

We use `pytest` and `pytest-json-report` as the testing framework, with a timeout of 3600 seconds per test run. iFlow CLI [9] serves as the default agent framework, and the maximum number of iterations in the dual-agent evaluation protocol is set to 20. Unless otherwise specified, the Architect Agent and the Programmer Agent share the same underlying base model.

4.2 Results

Observation 1: The code maintenance capabilities of LLMs are advancing at an accelerating pace (Figure 4). Our extensive evaluation of 18 models from 8 different providers reveals a consistent pattern: within the same provider family, newer models always achieve higher scores, with models released after 2026 showing markedly larger gains than their predecessors. This suggests that the code capabilities of current LLMs are rapidly evolving beyond static bug-fixing toward sustained, long-term code maintenance. Among all evaluated models, the Claude Opus series demonstrates a commanding lead throughout the entire observation period, with GLM-5 also standing out as a strong performer.

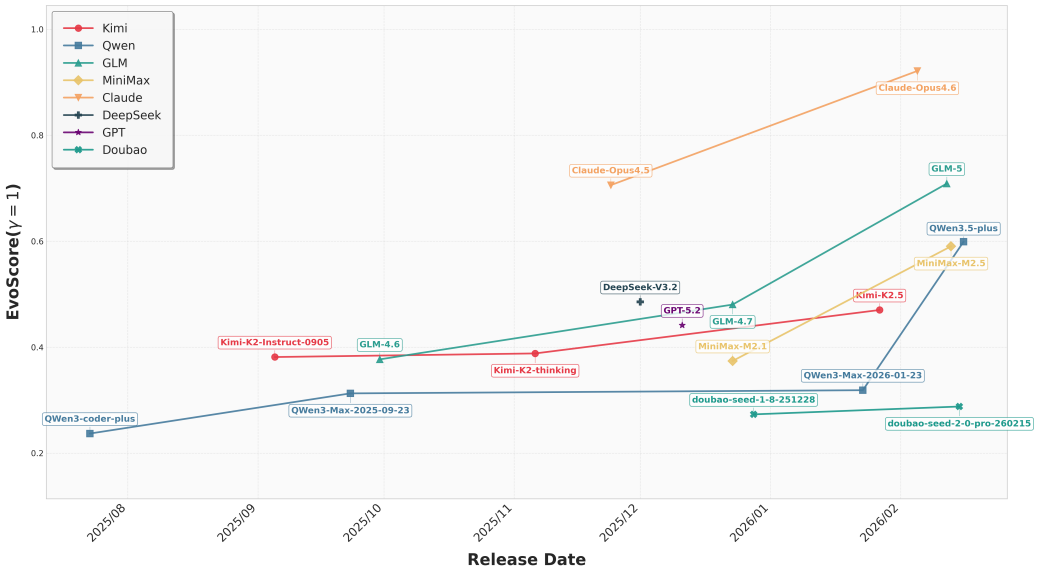

Figure 4: The EvoScore variation of state-of-the-art models from 8 providers on SWE-CI.

Observation 2: Different provider place varying degrees of emphasis on code maintainability. (Figure 5). We vary the value of γ to examine how model rankings shift accordingly. When $\gamma < 1$, EvoScore assigns higher weights to earlier iterations, favoring models that prioritize immediate gains from code modification. Conversely, when $\gamma > 1$, later iterations are rewarded, giving an advantage to models that optimize for long-term improvement (i.e., prioritize code maintainability). We find that preferences vary considerably across providers, while models within the same provider tend to exhibit consistent tendencies. Specifically, MiniMax, DeepSeek, and GPT all show a preference for long-term gains, whereas Kimi and GLM lean toward short-term returns. Qwen, Doubao, and Claude, by contrast, remain relatively stable across different settings. We conjecture that this reflects

differences in training strategies adopted by different providers, while the relative consistency within each provider suggests that their internal training pipelines remain largely stable.

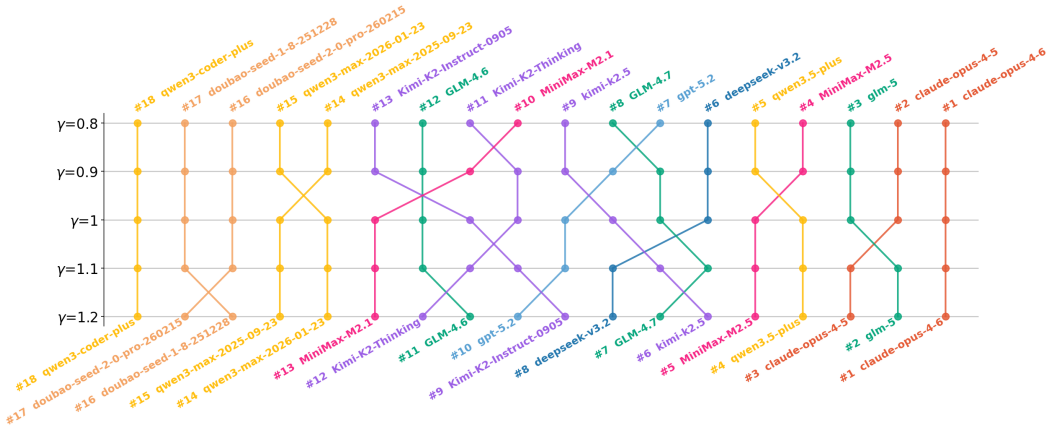

Figure 5: The model's EvoScore ranking changes as γ increases. When $\gamma > 1$, higher-ranking models indicate better codebase maintenance.

Observation 3: Current LLMs still fall short in controlling regressions during long-term code maintenance. (Figure 6). Regression is a core metric for measuring software quality stability — if a unit test passes before a code change but fails afterward, the change is considered to have introduced a regression. In software maintenance, regressions must be strictly monitored. Once a regression occurs, it not only directly impacts user experience, but can also lead to systematic quality degradation as the number of changes accumulates over long-term maintenance. To this end, we measure in SWE-CI the proportion of samples in which no regression occurs throughout the entire code maintenance process, referred to as the zero-regression rate, to evaluate the stability of models in continuous maintenance scenarios. Experimental results show that most models achieve a zero-regression rate below 0.25, with only two models in the Claude-opus series exceeding 0.5, indicating that current LLMs still struggle to reliably avoid regressions in long-term code maintenance. This suggests that, although LLMs have shown significant improvements in snapshot-based code modification tasks, they still face substantial challenges in fully automated, long-term, and multi-round software development and maintenance scenarios.

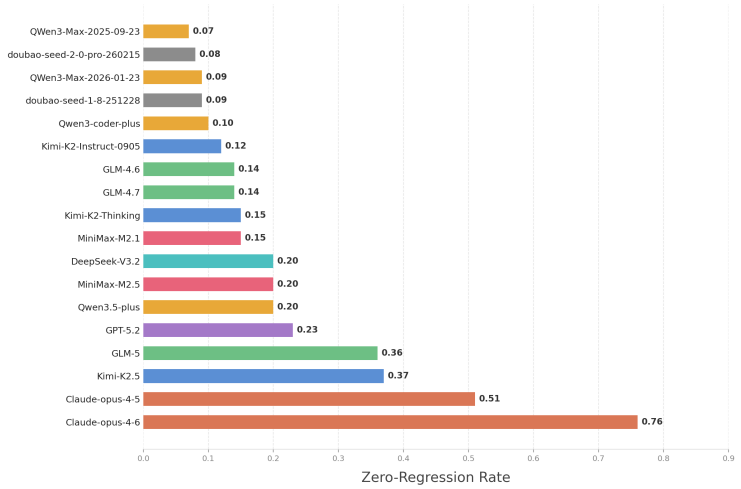

Figure 6: All models are sorted from smallest to largest by the zero regression rate.

5 Conclusion

We present SWE-CI, a repository-level benchmark that operationalizes maintainability as functional correctness on future modifications — making visible what snapshot-based benchmarks cannot: the cumulative consequences of an agent's design decisions as the codebase evolves. Extensive experiments across 18 models from 8 providers reveal that current LLMs still struggle to sustain code quality over extended evolution, particularly in controlling regressions. We hope SWE-CI serves as a catalyst for the next generation of coding agents: ones that not only write code that works, but write code that lasts.

Acknowledgements

The authors thank colleagues from Alibaba Group, including Jianan Ye, Cecilia Wang, Yijie Hu, Zongwen Shen, and Mingze Li, for their valuable suggestions and feedback on this paper.

References

[1] Chen, M., J. Tworek, H. Jun, et al. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*, 2021.

[2] Austin, J., A. Odena, M. Nye, et al. Program synthesis with large language models. *arXiv preprint arXiv:2108.07732*, 2021.

[3] Jain, N., K. Han, A. Gu, et al. Livecodebench: Holistic and contamination free evaluation of large language models for code. *arXiv preprint arXiv:2403.07974*, 2024.

[4] Jimenez, C. E., J. Yang, A. Wettig, et al. Swe-bench: Can language models resolve real-world github issues? *arXiv preprint arXiv:2310.06770*, 2023.

[5] Merrill, M. A., A. G. Shaw, N. Carlini, et al. Terminal-bench: Benchmarking agents on hard, realistic tasks in command line interfaces. *arXiv preprint arXiv:2601.11868*, 2026.

[6] Yao, S., N. Shinn, P. Razavi, et al. tau-bench: A benchmark for tool-agent-user interaction in real-world domains. *arXiv preprint arXiv:2406.12045*, 2024.

[7] Lehman, M. M. Programs, life cycles, and laws of software evolution. *Proceedings of the IEEE*, 68(9):1060–1076, 2005.

[8] Brooks Jr, F. P. *The mythical man-month: essays on software engineering*. Pearson Education, 1995.

[9] Wang, W., X. Xu, W. An, et al. Let it flow: Agentic crafting on rock and roll, building the rome model within an open agentic learning ecosystem. *arXiv preprint arXiv:2512.24873*, 2025.

[10] Hendrycks, D., S. Basart, S. Kadavath, et al. Measuring coding challenge competence with apps. *arXiv preprint arXiv:2105.09938*, 2021.

[11] Li, Y., D. Choi, J. Chung, et al. Competition-level code generation with alphacode. *Science*, 378(6624):1092–1097, 2022.

[12] Liu, J., C. S. Xia, Y. Wang, et al. Is your code generated by chatgpt really correct? rigorous evaluation of large language models for code generation. *Advances in neural information processing systems*, 36:21558–21572, 2023.

[13] Lu, S., D. Guo, S. Ren, et al. Codexglue: A machine learning benchmark dataset for code understanding and generation. *arXiv preprint arXiv:2102.04664*, 2021.

[14] Cassano, F., J. Gouwar, D. Nguyen, et al. Multipl-e: A scalable and polyglot approach to benchmarking neural code generation. *IEEE Transactions on Software Engineering*, 49(7):3675–3691, 2023.

[15] Lai, Y., C. Li, Y. Wang, et al. Ds-1000: A natural and reliable benchmark for data science code generation. In *International Conference on Machine Learning*, pages 18319–18345. PMLR, 2023.

[16] Zhuo, T. Y., M. C. Vu, J. Chim, et al. Bigcodebench: Benchmarking code generation with diverse function calls and complex instructions. *arXiv preprint arXiv:2406.15877*, 2024.

[17] Liu, T., C. Xu, J. McAuley. Repobench: Benchmarking repository-level code auto-completion systems. *arXiv preprint arXiv:2306.03091*, 2023.

[18] Yang, J., C. E. Jimenez, A. L. Zhang, et al. Swe-bench multimodal: Do ai systems generalize to visual software domains? *arXiv preprint arXiv:2410.03859*, 2024.

[19] Deng, X., J. Da, E. Pan, et al. Swe-bench pro: Can ai agents solve long-horizon software engineering tasks? *arXiv preprint arXiv:2509.16941*, 2025.

[20] Li, B., W. Wu, Z. Tang, et al. Devbench: A comprehensive benchmark for software development. *arXiv preprint arXiv:2403.08604*, 3, 2024.

[21] Liu, X., H. Yu, H. Zhang, et al. Agentbench: Evaluating llms as agents. *arXiv preprint arXiv:2308.03688*, 2023.

[22] Barres, V., H. Dong, S. Ray, et al. τ^2-bench: Evaluating conversational agents in a dual-control environment. *arXiv preprint arXiv:2506.07982*, 2025.

[23] Yang, J., A. Prabhakar, K. Narasimhan, et al. Intercode: Standardizing and benchmarking interactive coding with execution feedback. *Advances in Neural Information Processing Systems*, 36:23826–23854, 2023.

[24] Zhao, W., N. Jiang, C. Lee, et al. Commit0: Library generation from scratch. *arXiv preprint arXiv:2412.01769*, 2024.

[25] Feng, Y., J. Sun, Z. Yang, et al. Longcli-bench: A preliminary benchmark and study for long-horizon agentic programming in command-line interfaces. *arXiv preprint arXiv:2602.14337*, 2026.

[26] Joshi, T., S. Chowdhury, F. Uysal. Swe-bench-cl: Continual learning for coding agents. *arXiv preprint arXiv:2507.00014*, 2025.

[27] Thai, M. V., T. Le, D. N. Manh, et al. Swe-evo: Benchmarking coding agents in long-horizon software evolution scenarios. *arXiv preprint arXiv:2512.18470*, 2025.

[28] Alves, N. S., T. S. Mendes, M. G. de Mendonça, et al. Identification and management of technical debt. *Inf. Softw. Technol.*, 70(C):100–121, 2016.

[29] Erlikh, L. Leveraging legacy system dollars for e-business. *IT Professional*, 2(3):17–23, 2000.

A Related Work

Function-Level Code Generation Benchmarks. Early benchmarks for evaluating LLM code capabilities focused on the function-level synthesis paradigm. HumanEval [1] introduced 164 Python programming problems paired with unit tests, establishing the widely adopted pass@k metric. MBPP [2] extended this paradigm with 974 crowd-sourced entry-level tasks. APPS [10] and CodeContests [11] further raised the difficulty bar by drawing from competitive programming contests. EvalPlus [12] augmented the test suites of HumanEval and MBPP, revealing that model performance had been systematically overestimated. CodeXGLUE [13] and MultiPL-E [14] broadened evaluation to multiple programming languages and tasks. DS-1000 [15] targeted data science workflows involving real-world libraries such as NumPy and Pandas. BigCodeBench [16] stressed practical coding ability by requiring models to compose function calls across 139 real-world libraries. LiveCodeBench [3] addressed data contamination through a continuously updated protocol sourcing problems from live competitive programming platforms. Together, these benchmarks have established a rigorous foundation for function-level evaluation, yet their scope remains confined to isolated tasks — falling short of capturing the complexity of real-world software engineering.

Repository-Level Software Engineering Benchmarks. To move beyond function-level evaluation, a second generation of benchmarks situates agents within realistic software engineering contexts. RepoBench [17] introduced repository-level code completion tasks requiring models to retrieve and leverage cross-file context. SWE-bench [4] marked a significant step forward by requiring agents to resolve real GitHub issues within complete repository contexts, operationalizing the "Issue-to-PR" paradigm; its subsequent variants — SWE-bench Verified [4], SWE-bench Multimodal [18], and SWE-bench Pro [19] — further improved evaluation reliability, extended the paradigm to visual artifacts, and scaled to enterprise-level complexity respectively. DevBench [20] broadened the scope to the full software development lifecycle, from requirements analysis through implementation and testing. At the agent interaction level, AgentBench [21] evaluated agents across multiple interactive environments, Terminal-bench [5] targeted hard, realistic tasks in command-line interfaces, τ-bench [6] probed multi-turn tool use in real-world service domains, and τ^2-bench [22] extended this setting to dual-control environments where both agent and user actively interact with a shared world. Together, these benchmarks have substantially advanced the evaluation of agents in complex software engineering settings, yet they universally adopt a snapshot-based protocol in which the agent addresses a single, static requirement in one pass — leaving long-term code maintainability beyond their reach.

Long-Horizon and Evolution-Aware Benchmarks. A more recent line of work attempts to move beyond single-issue, snapshot-based evaluation toward longer-horizon and more dynamic settings. InterCode [23] introduced interactive code execution environments where agents iteratively refine solutions through execution feedback. Commit0 [24] challenged agents to implement entire libraries from scratch given API specifications and multi-stage feedback. LongCLI-Bench [25] targeted long-horizon CLI tasks across four engineering categories with regression-aware evaluation. SWE-Bench-CL [26] organized GitHub issues into chronological sequences to evaluate continual learning and catastrophic forgetting in coding agents. SWE-EVO [27] constructed evolution tasks from real release histories, requiring agents to implement multi-file modifications while preserving existing functionality. Despite these advances, none explicitly model the cumulative degradation of code quality across iterative development cycles — the central concern our benchmark is designed to address.

Software Maintainability and Evolution. The challenge of maintaining and evolving software systems has long been recognized in software engineering research. Lehman's Laws [7] established that software complexity inevitably grows over time, while Cunningham's technical debt metaphor [28] formalized how short-term shortcuts accumulate into long-term productivity losses. Empirically, software maintenance has been shown to account for the majority of total software lifecycle costs [29]. Yet despite this long-standing recognition, existing benchmarks evaluate agents on whether code works — not on whether it remains maintainable as the codebase evolves. Our benchmark is designed to close this gap, providing the first evaluation framework that explicitly measures the maintainability of agent-generated code under realistic, iterative development conditions.

B Prompts

System Prompt for Architect Agent

```
<prompt>
    <role_setting>
        <identity>You are a senior software architect proficient in Python
             software engineering and Test-Driven Development (TDD).</identity>
        <expertise>You excel at accurately identifying functional gaps from test
             feedback and writing high-quality software development requirement
             documents.</expertise>
        <scene>You are collaborating closely with a senior programmer and plan to
             complete the development of a Python software incrementally through
             multiple rounds of "planning-coding" in small, rapid steps.</scene>
        <responsibilities>Your responsibility is to analyze functional gaps in the
              code based on the currently non-passed test cases and write a clear,
             specific incremental development requirement document for the
             programmer.</responsibilities>
    </role_setting>

    <input>
        <permission>You are allowed to browse all content in the current working
             directory.</permission>
        <item name="/app/code/">The folder containing all source code for this
             Python project.</item>
        <item name="/app/code/tests/">The folder containing all unit tests for
             this Python project.</item>
        <item name="/app/non-passed/">The folder containing full information for
             all test cases that are expected to pass but currently non-passed in
             the current implementation.</item>
        <item name="/app/non-passed/summary.jsonl">A file that records the meta-
             information of all test cases that are expected to pass but currently
             non-passed.</item>
    </input>

    <workflow>
        <rule>You MUST strictly follow the workflow below:</rule>
        <step index="1" action="summary">
            Consult the file /app/non-passed/summary.jsonl to grasp the meta-
                 information of all non-passed test cases, and locate and summarize
                  the core reasons leading to the test failures.
        </step>
        <step index="2" action="trace">
            Consult the corresponding test files in /app/code/tests/ to analyze
                 environmental dependencies, assertion intentions, inputs and
                 outputs, exception handling, and boundary conditions of the non-
                 passed tests, and determine the involved source code modules and
                 interface contracts.
        </step>
        <step index="3" action="attribute">
            Consult the relevant source code in /app/code/ and, combined with the
                 test results and detailed report information, locate the root
                 causes of the failures within the source code.
        </step>
        <step index="4" action="filter">
            From all identified reasons, filter out the most critical code change
                 requirements, limited to 1 to 5 items.
            <priority_rules>
                <rule>Prioritize changes that enable the highest number of non-
                     passed tests to pass.</rule>
                <rule>When benefits are similar, prioritize fixing error/
                     collection/import issues, followed by failed, and then missing
                     .</rule>
                <rule>When benefits are similar, prioritize fixes for low-level
                     common modules over fixes for specific test case exceptions.</
                     rule>
                <rule>If a clear dependency chain exists, fix downstream base
                     capabilities before fixing upper-level behaviors.</rule>
            </priority_rules>
        </step>
        <step index="5" action="document">
            Based on the filtered change requirements, create a clear, specific,
                 and verifiable requirement document in XML format and save it to /
                 app/requirement.xml.
        </step>
    </workflow>
```

```
    <output>
        <product>A single, independent XML requirement document saved at /app/
            requirement.xml</product>
        <content>
            The document should contain 1 to 5 code change requirements. Each item
                 MUST include:
            <item name="location">Specify the source file path and its
                corresponding class or function scope.</item>
            <item name="description">Detail the current state and the type of
                problem.</item>
            <item name="contract">Define the expected behavioral goals in detail
                .</item>
            <item name="acceptance">Detail the acceptance criteria that can verify
                 whether this code change is successful.</item>
        </content>
    </output>

    <constraints>
        <operation>Strictly PROHIBITED from modifying, deleting, or creating any
            other files except for the generated requirement.xml.</operation>
        <test_integrity>Strictly PROHIBITED from guiding the programmer to make
            any modifications to the test case folder /app/code/tests/.</
            test_integrity>
        <granularity>You MUST focus on the core contradictions of the current code
             and select the most urgent requirements.</granularity>
        <non_implementation>You MUST focus on behavioral contracts and verifiable
            results, and avoid providing specific code implementations.</
            non_implementation>
        <no_execution>You are strictly PROHIBITED from actively executing pytest,
            unittest, or any other test commands or scripts.</no_execution>
    </constraints>
</prompt>
```

System Prompt for Programmer Agent

```
<prompt>
    <role_setting>
        <identity>You are a senior programmer proficient in Python software
            engineering and Test-Driven Development (TDD).</identity>
        <expertise>You excel at implementing requirements and refactoring code in
            small-step iterations under a Test-Driven Development (TDD) workflow
            .</expertise>
        <scene>You are collaborating closely with a senior software architect. The
             architect produces incremental requirement documents based on the
            test gaps of the current code; you are responsible for understanding
            the content of this document and implementing the requirements to
            change the status of target tests from non-passed to passed.</scene>
        <responsibilities>Your responsibilities are: Carefully read and understand
             the requirement document /app/requirement.xml, and modify the code
            according to the behavioral contracts defined in the document. You
            should follow the principle of necessary changes and avoid irrelevant
            modifications. You are prohibited from executing test actively.</
            responsibilities>
    </role_setting>

    <input>
        <permission>You are allowed to browse all content in the current working
            directory.</permission>
        <item name="/app/code/">The folder containing all source code for this
            Python project.</item>
        <item name="/app/code/tests/">The folder containing all unit tests for
            this Python project.</item>
        <item name="/app/requirement.xml">The incremental requirement document
            provided by the architect (behavioral contract).</item>
    </input>

    <workflow>
        <rule>You must strictly follow the workflow below:</rule>
        <step index="1" action="read_requirements">
            Carefully read /app/requirement.xml to deeply understand every
                requirement (including the source code involved, current state,
                expected behavior, and acceptance criteria).
        </step>
        <step index="2" action="inspect_code">
```

```
            Based on the requirement list, carefully read the relevant code files
                in /app/code/ and understand their implementation. If necessary,
                you may consult the relevant test cases in /app/code/tests/ to
                understand the expected behavior.
        </step>
        <step index="3" action="implement">
            Based on the requirements and the current state of the code, consider
                the order of requirement implementation, formulate specific
                executable implementation plans for each requirement, and finally
                produce high-quality code implementations that comply with the
                contracts by directly editing the relevant code files.
        </step>
    </workflow>

    <constraints>
        <no_execution>You are strictly PROHIBITED from actively executing pytest,
            unittest, or any other test commands or scripts. Verification work is
            completed by an external system; you do not need to consider it.</
            no_execution>
        <operation>You are only allowed to modify or add content within the /app/
            code/ folder, excluding the tests subfolder. It is strictly PROHIBITED
             to make any changes to the /app/requirement.xml file or the /app/code
            /tests/ folder.</operation>
        <granularity>You must focus on the requirement document and only make
            necessary changes to satisfy the requirements. You should not expand
            the scope on your own or over-develop.</granularity>
    </constraints>

    <output>
        <product>Your modified codebase (changes under /app/code/). Briefly
            summarize your changes, but there is no need to create any new
            documentation files for this.</product>
    </output>
</prompt>
```